Assignment #4 - Implementation Testing

Part A

A patient record system is a crucial operational element of healthcare facilities, containing information of thousands of patients. The nature of the patient records are that they have to be accurate and stored securely on a long-term basis, and protected from data inconsistencies, unintentional duplication or its alteration. Any flaws or quality issues in this database may directly impact the patients, and may even lead to serious harm. For this reason, SPRS has to be thoroughly tested to ensure that all the system operations are functioning correctly under real-world scenarios, and the system requirements are fulfilled.

Our Simple Patient Record System is based on five mandatory requirements:

- **(REQ-01)** The system shall allow the creation of a new patient record with a name and a unique Patient ID.
- **(REQ-02)** The system shall retrieve a patient's record using their unique Patient ID.
- **(REQ-03)** The system shall ensure data integrity by preventing the modification of a Patient ID once it has been assigned.
- **(REQ-04)** The system shall allow updating a patient's name using their Patient ID, while keeping the Patient ID unchanged.
- **(REQ-05)** The system shall allow deleting a patient record using their Patient ID. If the ID does not exist, the system shall display an appropriate error message.

To ensure these requirements are met and the system functions correctly, it requires rigorous testing of all its components. To test this system, Unit Testing, Component Testing, and Test Coverage Analysis will be applied to test the functionality of the Simple Patient Record System. Also, SPRS will also be tested to maintain different test cases and operations, such as retrieving information from the previously deleted patient, and testing invalid scenarios.

To implement its test plans, automated testing and unittest were used.

Part B

Requirements Traceability Matrix

Req and Test Case IDs	UT-01	UT-02	UT-03	UT-04	UT-05	UT-06	CT-01	CT-02
REQ-01: Creation	X						X	X
REQ-02: Retrieval		X	X				X	X
REQ-03: Immutability				X			X	
REQ-04: Update				X	X		X	
REQ-05: Delete						X		X

Part C

Unit Test Cases

Unit Test Case 1 - Normal Case

Test Case ID	UT-01
Input	1. register_patient(“John Smith”)
Expected Output	1. Patient ID = “P-101” 2. Patient ID will exist in the patient registry 3. Patient’s record will look like {“patient_id”:”P-101”,”name”:”John Smith”}
Actual Output	1. Patient ID = “P-101” 2. Patient ID does exist in the patient registry 3. Record is structured properly.
Pass/Fail	Pass
Purpose	1. To verify the patient registration generates the unique and stores the record correctly and in the right format

Unit Test Case 2 - Normal Case

Test	UT-02

Case ID	
Input	1. register_patient(“John Smith”) 2. get_patient(patient_ id)
Expected Output	1. Correct patient record returned. 2. Record should be {“patient_id”: “P-101”, “name”: “John Smith”}
Actual Output	1. Correct patient record returned. 2. Record should be {“patient_id”: “P-101”, “name”: “John Smith”}
Pass/Fail	Pass
Purpose	2. To verify get_patient() gets the correct dict record with the correct assigned Patient ID.

Unit Test Case 3 - Edge Case

Test Case ID	UT-03
Input	get_patient(“P-000”) since P-000 does not exist
Expected Output	Should return None
Actual Output	Returns None
Pass/Fail	Pass
Purpose	Verifies that the retrieval of the patients that don’t exist return nothing and not an error

Unit Test Case 4 - Boundary Case

Test Case ID	UT-04
Input	1. register_patient(“John Smith”) 2. update_patient_name(patient_id, “Jim Smith”)
Expected Output	1. Name should be changed to Jim Smith 2. Patient ID does NOT change

Actual Output	1. Name is correct (Jim Smith) 2. Patient ID stays the same
Pass/Fail	Pass
Purpose	Verifies if the Patient ID remains immutable during name updates

Unit Test Case 5 - Invalid Input Case

Test Case ID	UT-05
Input	update_patient_name(“P-000”, “Jane Doe”) since P-000 doesn’t exist
Expected Output	Should return None
Actual Output	Returns None
Pass/Fail	Pass
Purpose	Verifies updating to a non-existent patient ID returns none with no error

Unit Test Case 6 - Invalid Input Case

Test Case ID	UT-06
Input	1. register_patient(“Delete Check”) 2. delete_patient(patient_id) 3. get_patient(patient_id) 4. delete_patient(“P-000”)
Expected Output	1. Delete valid patient returns true 2. The deleted patient record is no longer retrievable. 3. Can not delete a patient that does not exist and return False
Actual Output	1. Deleting valid patient returns true 2. The deleted patient record is no longer retrievable. 3. Can not delete a patient that does not exist and returned False

Pass/Fail	Pass
Purpose	Verifies successful delete and handling of invalid patient ID

Component Test Cases

Component Test Case 1 - Valid Scenario

Test Case ID	CT-01
Input	1. Register a new patient: “Jane Doe” 2. Update patient name to “Jane Smith” 3. Retrieve patient ID
Expected Output	1. Patient registration successful under generated ID 2. “Jane Doe” -> “Jane Smith” 3. New Patient name and information is under generated ID
Actual Output	1. Patient Jane Doe registration successful their assigned ID is: P-101 2. Name Changed Updated Info: {'patient_id': 'P-101', 'name': 'Jane Smith'} 3. Patient P-101's info: {'patient_id': 'P-101', 'name': 'Jane Smith'}
Pass/Fail	Pass
Purpose	1. To simulate a common, workflow-based interaction with the Simple Patient Record System: *Register a new patient → Update Patient information → Retrieve Patient information* 2. Ensure that REQ-01, REQ-02, REQ-03, and REQ-04 are fulfilled during workflow-based interaction: a. (REQ-01): generate an unique Patient ID and store the patient record in it. b. (REQ-02): retrieve a patient's record using their unique Patient ID. c. (REQ-03): make assigned patient ID unchangeable to ensure data integrity. d. (REQ-04): update a patient’s name using their Patient ID.

Component Test Case 2 - Failure/Defect scenario

Test Case ID	CT-02
Input	1.Register a new patient: “Jo Johnson”

	2. Delete patient ID 3. Retrieve patient ID
Expected Output	1. Patient registration successful under generated ID 2. Patient was successfully deleted 3. There is no Patient data under the assigned ID
Actual Output	1. Patient Jo Johnson registration successful their assigned ID is: P-101 2. Patient deleted 3. Patient does not exist
Pass/Fail	Pass
Purpose	1. To simulate a failure/defect based interaction scenario with the Simple Patient Record System: *Register a new patient → Delete Patient → Retrieve Patient information* 2. Ensure that REQ-01, REQ-02, REQ-05 and error handling system are fulfilled during a workflow-based interaction: a. (REQ-01): generate an unique Patient ID and store the patient record in it b. (REQ-02): retrieve a patient's record using their unique Patient ID. c. (REQ-05): deletion of a patient record using their Patient ID. d. Error handling to prevent data access after deletion.

Part D

I chose to implement our automated tests using unittest since it is built into Python's standard library and it seemed to be very straightforward from the research I did to learn how to use it. Our goal was obviously to verify that our PatientRegistry class safely meets all requirements specified in the last assignment. This includes patient registration, patient retrieval, name update without changing ID, patient deletion, and again protecting the ID. Since we used a dictionary to simulate our database, we were able to easily check the state of our dictionary after each operation to make sure our system performed properly. I also had to add tests for my helper functions list_all_patients and get_patients_name. I had to add one check anyway to get to the 85% coverage mark, so I went ahead and added checks for the other function to get us up to 100% coverage.

Our testing strategy obviously included the unit and component tests laid out above in section C. Our unit tests were used to validate the individual methods, and to cover normal, boundary, edge, and invalid cases. The component tests were used to

verify the full project workflow in both valid and invalid cases. One of our tests starts at registering a patient, then to updating patient info, and then to retrieving the patient's information, while also verifying the ID is unchanged. This is how we handled REQ-03, since the patient name is changed but ID is not. The other component test starts with registering a patient and then moves to deleting that patient followed by a confirmation check that the patient was actually deleted. Both of these tests ensure that the system works properly no matter what operation the user calls. I also tested invalid ID handling directly by using ID “P-000” for update, delete, and retrieval of patients. In our registry class, invalid updates and retrievals return None, and invalid deletion returns false. These returns are what trigger our safe handling of invalid user input, which is just simply displaying a console message telling the user that the patient does not exist.

Our final test pass was 12 out of 12 and the coverage analysis was performed using the coverage tool just like you required. The resulting coverage was 100% for patient_registry.py and 99% overall. Obviously coverage does not guarantee that a program will be completely error free but it definitely shows that all core logic was checked during testing.

Overall, testing our code was much easier than I had anticipated it being. As well I would say our approach was effective because it tested both individual methods and our full workflow. I was also surprised at how easy it was to call these individual methods since like I said I have no prior experience using this test suite.

Part E

```
(.venv) PS C:\Users\KodyD\PycharmProjects\SimplePatientRecordSystem> python -m coverage run -m unittest test_patient_registry.py
............
----------------------------------------------------------------------
Ran 12 tests in 0.005s

OK
```

```
(.venv) PS C:\Users\KodyD\PycharmProjects\SimplePatientRecordSystem> python -m coverage report -m
Name                        Stmts   Miss  Cover   Missing
-------------------------------------------------------
patient_registry.py            33      0   100%
test_patient_registry.py       67      1    99%   124
-------------------------------------------------------
TOTAL                         100      1    99%
(.venv) PS C:\Users\KodyD\PycharmProjects\SimplePatientRecordSystem>
```

Part F

https://github.com/KGraham38/SimplePatientRecordSystem

Part G

Daniyar Alimkhanov:

- **Part A**
- **Part C**

<u>Seth Wojcik:</u>

- **Part B**
- **Part C**

<u>Kody Graham:</u>

- **Part D**
- **Part E**
- **Part F**
- **Part G**